CS24309 — THEORY (3.0 CR) | MODULE II NOTES | BIT MESRA | NEP SCHEME

Module 2: Requirements & DFD Analysis

IEEE 830 SRS Standard, Use Case Specifications, Data Flow Diagrams & Data Dictionaries

Module II: Requirements Engineering, Analysis Modeling & Structured Specification

1. Requirements Engineering Process (Elicitation, Analysis, Specification, Validation)
2. Functional Requirements vs. Non-Functional Requirements (FURPS+ Model)
3. IEEE 830 Standard Format for Software Requirements Specification (SRS)
4. Characteristics of High-Quality SRS Documents (Traceable, Unambiguous, Verifiable)
5. Use Case Modeling: Actors, Use Cases, `<>` vs. `<>` Relationships
6. Structured Analysis: Data Flow Diagrams (DFD) — 4 Core Geometric Notations
7. DFD Leveling Hierarchy: Context Diagram (Level 0), Level 1 & Level 2 Explosions
8. DFD Balancing Invariants & Common Modeling Errors (Black Holes, Miracles, Gray Holes)
9. Data Dictionary Architecture: BNF Metadata Syntax & Composite Data Elements
10. Entity-Relationship (ER) Modeling: Entities, Attributes, Cardinalities & Keys
11. Decision Tables & Structured English for Complex Business Logic Modeling
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

Topic 2 – 4: Requirements Engineering & The IEEE 830 SRS Standard

A **Software Requirements Specification (SRS)** is a formal contract between the client and the development organization describing the complete functional behavior and operational constraints of the system.

SRS Section (IEEE 830)	Mandatory Content & Description	Key Invariants & Purpose
1. Introduction	Purpose of the system, project scope, definitions, acronyms, and references.	Sets project boundary; prevents scope creep.
2. Overall Description	Product perspective, user classes, operating environment, design constraints, assumptions.	Provides high-level context without dictating low-level implementation details.
3. Specific Requirements	Detailed Functional Requirements, External Interface Requirements (UI, HW, SW), Non-Functional Requirements (Performance, Security, Reliability).	Must be 100% **Unambiguous, Complete, Consistent, Verifiable (Testable), and Traceable**.

Topic 6 – 8: Data Flow Diagrams (DFD) & Balancing Rules

DFD Symbol	DeMarco & Yourdon	Gane & Sarson	Functional Meaning
Process	Circle (Bubble)	Rounded Rectangle	Transforms input data flows into output data flows.
Data Flow	Directed Arrow	Directed Arrow	Pipeline along which packets of data travel.
Data Store	Two Parallel Lines	Open Rectangle	Repository of data at rest (files, DB tables).
External Entity	Rectangle	Shaded Rectangle	Source or Sink of data outside system boundary.

DFD Balancing & Consistency Invariants

- **Conservation of Data:** A process cannot output data that was not present in or computable from its input data.
- **Black Hole Error:** A process that has inputs but zero outputs.
- **Miracle Error:** A process that generates outputs with zero inputs.
- **Gray Hole Error:** A process where outputs require data not supplied by its inputs.
- **Level Balancing:** All input data flows and output data flows entering/leaving a parent process bubble in DFD Level N must exactly match the external data flows in its child Level $N + 1$ diagram.

🏛 Top BIT Mesra Exam Questions & Answers (Module II)

Q1. Differentiate between `<>` and `<>` relationships in Use Case Diagrams with examples. (8 Marks)

1. **`<>` (Mandatory Shared Behavior):** The base use case unconditionally and explicitly invokes the included use case every single time it executes. Used to factor out common repeated subroutines.
Example: `Withdraw Cash` $\xrightarrow{<>}$ `Authenticate User PIN`.
2. **`<>` (Optional / Conditional Behavior):** The extending use case executes only if a specific extension condition holds true at designated extension points in the base use case. The base use case is completely functional on its own without knowing about the extension.
Example: `Calculate Premium` $\xleftarrow{<>}‘*ApplySeniorCitizenDiscount*‘(*onlyif*\text{age}\ge 60$).